

VecindApp.

Alumno: Marius Ion Dodu

Tutor: Emilio Saurina

Semestre: 2S2526

Índice de la presentación

01 Motivación y objetivos

02 Estado del arte y nicho

03 Metodología y stack

04 Análisis, diagramas y
diseño

05 Demostración en directo

06 Pruebas, conclusiones y
vías futuras

1. El problema que queremos resolver

iLERNA.

«Todos hemos necesitado ayuda con un arreglo en casa o con un trámite... y muchos tenemos habilidades y tiempo que podríamos ofrecer a quien lo necesita en nuestro barrio.»

NECESIDADES SIN CUBRIR

28%

de los hogares en España en riesgo de pobreza o exclusión social (INE, 2024)

RECURSOS DISPONIBLES



vecinos con **habilidades y tiempo** disponibles para ayudar a otros en el barrio

Falta el canal que los conecte.

La solución: VecindApp

Una aplicación Android que conecta vecinos para intercambiar servicios usando el tiempo como única moneda, con accesibilidad universal desde el minuto uno.

Hiperlocal

Conexión entre vecinos del mismo barrio, con publicaciones y categorías visuales.

Banco de tiempo

Una hora vale lo mismo para todos, sin dinero. Saldo inicial: 2h para empezar.

Accesible

TTS nativo + pictogramas ARASAAC. Accesibilidad activa, no pasiva.

Enfoque offline-first · Android nativo · Sin backend remoto

Objetivo general y arquitectura

OBJETIVO GENERAL

Desarrollar una aplicación Android nativa plenamente funcional que permita gestionar, publicar e intercambiar servicios vecinales mediante un saldo de horas, con una interfaz accesible para cualquier persona que la necesite.

PATRÓN ARQUITECTÓNICO

MVVM + Clean Architecture

Código escalable y mantenible, organizado en 4 capas: **data · domain · ui · common**

8 objetivos específicos



iLERNA.

OE-01

Base de datos local

Room con 4 entidades y relaciones FK

OE-02

Transacciones atómicas

Con validación de saldo

OE-03

TTS nativo

Lectura en voz alta accesible

OE-04

Valoraciones ARASAAC

Pictogramas en lugar de estrellas

OE-05

Dashboard gráfico

Historial con MPAndroidChart

OE-06

Autenticación local

Con nombres únicos

OE-07

Categorización visual

Escaparate con pictogramas dinámicos

OE-08

Notificaciones reactivas

Badge en el BottomNav

Veremos al final qué objetivos se cumplieron... y cuáles se superaron en alcance.

Conceptos clave

CONCEPTO 1

Banco de tiempo

Mecanismo de economía colaborativa donde la moneda no es el dinero, sino el tiempo.

1 hora = 1 hora para todos

independientemente del servicio prestado.

CONCEPTO 2

Accesibilidad

Desde junio de 2025, el **Acta Europea de Accesibilidad** obliga a cumplir las WCAG 2.2 AA.

**Ya no es una buena práctica:
es obligación legal.**

VecindApp integra **TTS nativo** + **pictogramas ARASAAC** (12.000+ bajo CC, estándar internacional en Comunicación Aumentativa).

2. Estado del arte: plataformas comparadas

Nextdoor

Red vecinal potente

EE.UU., 11 países, 265K barrios. Financiada con capital riesgo y publicidad.

⚠ **NO es banco de tiempo**

¿Tienes sal?

España, 560 vecindarios

Red local potente en su momento.

⚠ **Cerró por problemas de monetización ética**

TimeOverflow

Software oficial

De la Asociación de Bancos de Tiempo.

⚠ **Portal web administrativo, no es app móvil**

TimeRepublik

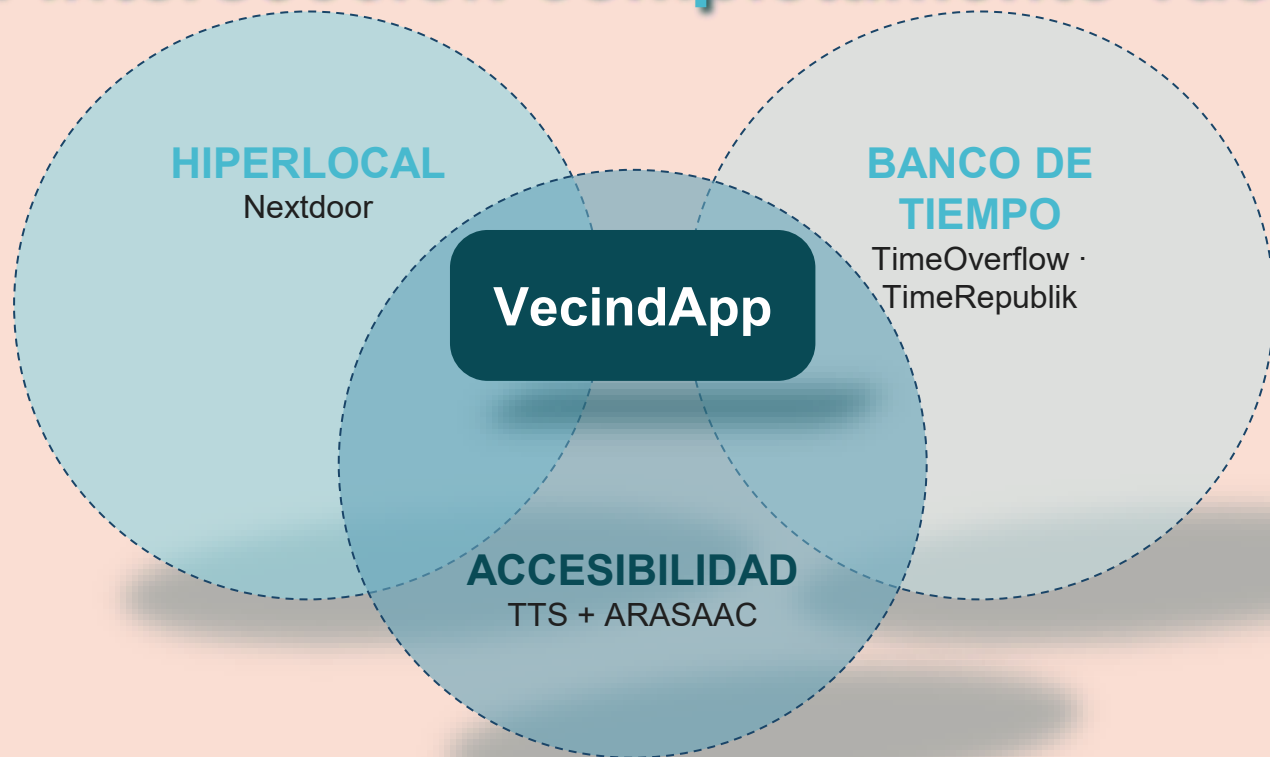
Red global

Con moneda propia «TimeCoins». Escalable.

⚠ **Sin componente hiperlocal**

Ninguna integra accesibilidad avanzada.

El nicho: intersección completamente vacía iLERNA.



Ninguna plataforma de banco de tiempo incorpora pictogramas o TTS. Ninguna app vecinal integra accesibilidad activa. **Para esta presentación VecindApp se ha preparado con enfoque offline-first. (Sin necesidad de internet)**

3. Metodología: Scrum adaptado a solitario

ADAPTACIÓN DE SCRUM

✓ Se conserva

- Sprints de duración fija
- Backlog priorizado

X Se descarta

- Daily (sin sentido en solitario)
- Retrospectiva en grupo

ESTRUCTURA EN 5 FASES

- 1 · Análisis inicial y estado del arte
 - 2 · Diseño de arquitectura y diagramas
 - 3 · Diseño visual y prototipado
 - 4 · Desarrollo por sprints (6 iteraciones)
 - 5 · Documentación y presentación
- MVP: login, escaparate, transacciones, valoraciones con TTS.*



4. Stack tecnológico

LENGUAJE

Kotlin 2.0.21

Recomendado oficialmente por Google



UI

XML + Fragments

No Jetpack Compose (según propuesta)



BASE DE DATOS

Room 2.8.4 ·

SQLite

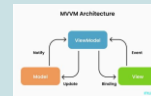
Filosofía offline-first



ARQUITECTURA

MVVM + Clean

4 capas: data · domain · ui · common



NAVEGACIÓN

Navigation 2.9.7

BottomNav de 4 pestañas



ASÍNCRONO

Coroutines + StateFlow

Programación reactiva



GRÁFICOS

MPAndroidChart 3.1.0

Librería terceros justificada



ACCESIBILIDAD

Android TTS + ARASAAC

Material Design 1.12.0



Compatible con Android 8.0+ → cubre >95 % del parque de dispositivos activos

5. Planificación y DAFO

ESTIMADO vs. REAL

Sprint 1 de arquitectura

+ tiempo: aprender MVVM + Clean a fondo costó más de lo estimado. Organizar estructura del proyecto

Sprint 2 & 3

Se redujo por avances en el S1.

Sprint 6 Alargado

El S6 se alargó por diferentes bugs y donde se tuvo que priorizar las tareas para el MVP

Sprint 7 Memorias y Setup

En S7 Ha tomado más tiempo por la documentación detallada y setup grabación con pocos recursos

FORTALEZAS

- Enfoque innovador: accesibilidad + banco de tiempo
- Arquitectura escalable

DEBILIDADES

- Ausencia de backend
- Sin sincronización entre dispositivos

OPORTUNIDADES

- Acta Europea de Accesibilidad
- WCAG 2.2 AA obligatorias

AMENAZAS

- Que Nextdoor incorpore esta funcionalidad

DAFO (destacados)

6. Análisis: casos de uso

ACTOR

Vecino

2 roles según contexto:

Comprador (gasta horas) ·

Vendedor (gana horas)

12

casos de uso

3 RELACIONES CLAVE

Permiten reutilizar comportamientos comunes y extender funcionalidad.

<<extend>>

CU-05 (TTS)

Extiende las pantallas de escaparate, detalle, valoración y perfil. Activación opcional.

<<extend>>

CU-11 → CU-10

Valorar con ARASAAC extiende «completar transacción». Regla unidireccional: solo el comprador valora.

<<include>>

Validación de saldo

Obligatoria en CU-10 (completar) y compartida con CU-09 (solicitar servicio).

Modelo de datos: 4 entidades con Room

Usuario

id (PK)
nombre único
saldo (horas)
barrio
avatar

Servicio

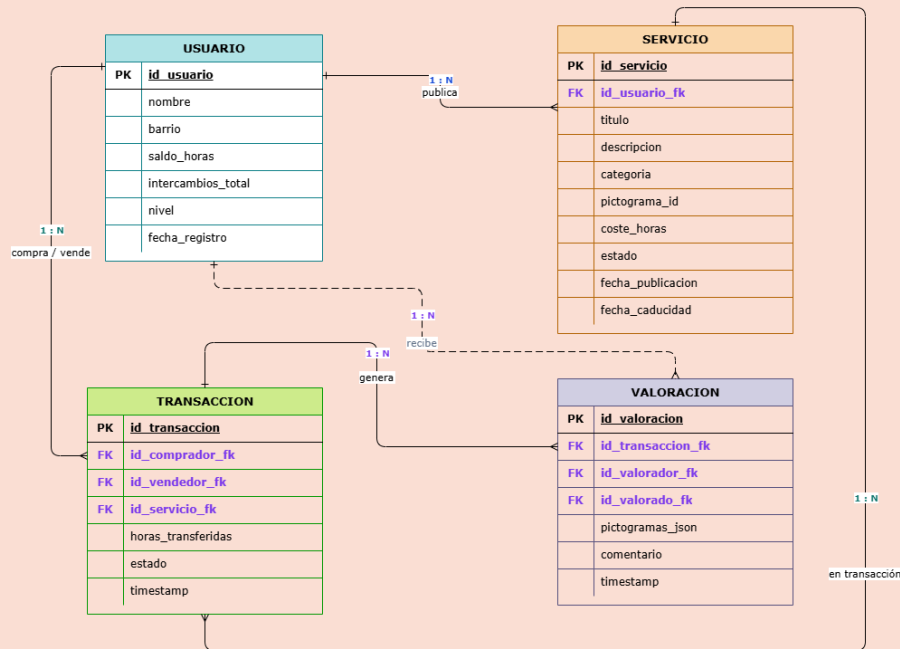
id (PK)
usuarioid (FK)
título
categoría
coste (h)

Transacción

id (PK)
servicioid (FK)
compradorId (FK)
estado
fecha

Valoración

id (PK)
transaccionId (FK)
pictogramas
comentario



Relaciones con claves foráneas · Valoración **unidireccional** del comprador al vendedor

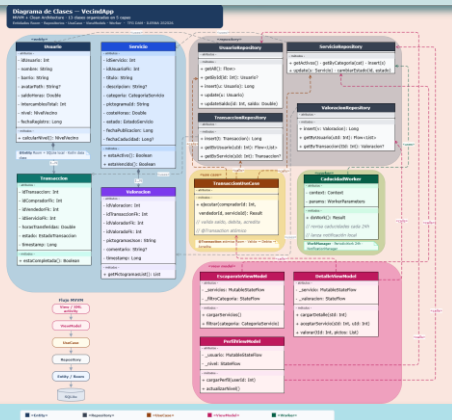
Arquitectura de clases: MVVM + Clean

iLERNIA.

ESCALADO DEL DISEÑO

13 → ~45

clases en 4 capas Clean



LAS 4 CAPAS

data · Room, DAOs, mapper

domain · casos de uso, interfaces

ui · Fragments, ViewModels

common · utilidades, extensiones



PRINCIPIO 1 · Responsabilidades aisladas
Los repositorios **no se comunican entre sí**. El ViewModel orquesta las llamadas.

PRINCIPIO 2 · Inversión de dependencias
Comunicación contra **interfaces de domain**, nunca contra implementaciones concretas.

PRINCIPIO 3 · Modelos de presentación
TransaccionUI · PerfilUI · EscaparateUI →
no contaminar las entidades Room con
lógica de UI.

7. Diseño visual

Login y Registro

Local, nombres únicos

Escaparate

Tarjetas con pictogramas de categoría

Crear / Detalle servicio

FAB, Slider de coste

Transacciones

Pendientes y completadas

Valoración ARASAAC

BottomSheet con pictogramas

Perfil

Saldo, nivel, servicios publicados

Historial

Gráfico MPAndroidChart

PALETA LIMPIA · ALTO CONTRASTE · MATERIAL DESIGN COMPONENTS

Elementos estándar: **BottomNavigationView · BottomSheet · Slider · Snackbar**

Recorrido de la demo (6 bloques)

iLERNA.

1 Registro y login

Nombre único, barrio, avatar.
Saldo inicial 5 h. Sesión persistente.

2 Escaparate + TTS

Pictogramas ARASAAC por categoría. Lectura en voz alta con un solo botón.

3 Publicar servicio

FAB → título, categoría, coste (Slider). LiveData reactivo. Integridad referencial.

4 Transacción atómica

Solicitar → validación de saldo → completar. Horas transferidas sin estados inconsistentes.

5 Valoración ARASAAC

BottomSheet con 9 pictogramas. Regla unidireccional: solo el comprador valora.

6 Perfil e Historial

Saldo, nivel, gráfico MPAndroidChart (ganadas vs. gastadas). Badge reactivo StateFlow.

Todo en tiempo real sobre dispositivo físico · sin conexión a internet



BLOQUE DEMO EN VIVO

Demostración en directo

Dispositivo físico · Android 16 · Offline-first

- ✓ 2 usuarios de prueba registrados (Marius y Amanda)
- ✓ Servicios registrados con anterioridad
- ✓ Añadir nuevo usuario (Irene)
- ✓ Publicar y valorar nuevo servicio
- ✓ Probar TTS
- ✓ Pantalla emitiendo al PC vía USB en tiempo real

8. Testing y bugs destacados

ENFOQUE DE TESTING

- Testing continuo durante el desarrollo
- Enfoque manual sobre **los 12 casos de uso**
- Prueba de humo documentada
registro → publicación → solicitud → transacción → valoración
- Bugs clasificados por severidad

BUG DIDÁCTICO DESTACADO

Sistema de valoración

Antes: bidireccional automático → generaba inconsistencias con la lógica real de un banco de tiempo.

Después: unidireccional voluntario, inspirado en BlaBlaCar. Solo el comprador valora, y solo si quiere.

Las pruebas no sólo cazan bugs técnicos; también validan decisiones de diseño.

9. Resultados y objetivos

8 / 8

OE cumplidos

3 OBJETIVOS SUPERADOS EN ALCANCE

TTS en 4 pantallas · Valoraciones unidireccionales voluntarias
· Notificaciones StateFlow reactivo

OBJETIVOS NO ALCANZADOS

WorkManager

Notificaciones programadas descartadas en Sprint 6 por tiempo. Pasa a vías futuras.

Firebase Firestore

Incompatible con la filosofía offline-first del MVP.

BALANCE GLOBAL

Muy positivo

El MVP cumple los objetivos y la arquitectura permite incorporar las vías futuras **sin refactorización profunda**.

10. Vías futuras

CORTO WorkManager

Notificaciones locales programadas. La entidad Servicio ya contempla fecha de caducidad.

TÉCNICO Mejoras técnicas

Tema oscuro, migraciones Room manuales, hasheo de contraseñas, auth biométrica.

ACCESIBILIDAD A11y avanzada

Integración completa con TalkBack, personalización del TTS, más pictogramas ARASAAC.

FUNCIONAL Más funcionalidad

Filtros en el escaparate (categoría, barrio, valoración) y mensajería directa.

LARGO PLAZO Backend Firestore

Sincronización real entre dispositivos sin perder la filosofía de privacidad.

VecindApp.

MVP funcional, accesible y escalable

¡Gracias!